



Sorokin School

Модуль №1 задание "Interrupt Me If You Can" [PDF]

Кодовое-название: Interrupt Me If You Can

Референс: *Catch Me If You Can*

История предисловие к заданию:

[=] Вы — агент, охотящийся за потоками, которые разбегаются по ядрам операционной системы, как хитрые беглецы. Планировщик — хаотичный дирижёр, меняющий порядок исполнения каждую миллисекунду.

Иногда вам удаётся поймать задачу и кинуть ей `interrupt()`, но чаще она ускользает в бесконечный контекст-свитч. В этой лаборатории вы сами проживёте жизнь потока — создадите, запустите, приостановите и остановите его.

В итоге вы познаете разницу между `run()` и `start()`, поймёте, зачем нужны `daemon`-потоки и как `volatile` и `synchronized` удерживают порядок в хаосе. Обуздаете хаос исполнения — потоки слушаются `interrupt()`, `volatile` и `synchronized`.

Цель

Закрепить: `start/run/join/sleep/interrupt`, состояния потоков, различие `user/daemon`, корректная остановка, `volatile` (видимость) vs `synchronized` (взаимное исключение). Логи должны демонстрировать **непредсказуемый порядок** и корректную обработку прерываний.

Задания

1) `StartVsRunDemo` — «старт против запуска»

Роль: показать `run()` (тот же поток) vs `start()` (новый поток).

Методы/поведение:

- `public static void main()` — создаёт три задания (наследование `Thread`, `Runnable`, лямбда), внутри потока добавить случайные задержки `sleep(10..80ms)`
- каждый лог должен содержать время, имя потока и сообщение
- в `main()` залогировать `currentThread().getName()` и `getState()` до/после/во время сна.

Ожидаем: в логах — перемешанный порядок и разные имена потоков для `start()`.

Проверка работы:

- Реализуйте `StartVsRunDemo.main()`
- проверьте логи, если запустить потоки через `run()` → тот же `main`-поток;
- проверьте логи, если запустить потоки через `start()` → новые имена потоков;
- посмотрите на состояния через `getState()`

2) `LoopWorker` — воркер с корректной остановкой

Роль: долгоживущий цикл с кооперативным завершением через `interrupt()`.

Конструктор: `LoopWorker(String name)` — задаёт имя потока.

Методы/поведение:

- `void start()` — запускает внутренний `Thread` (user-thread).
- `void stopAsync()` — вызывает `thread.interrupt()`.
- Цикл: раз в ~200–300 мс инкремент локального счётчика, лог состояния. Если пойман `InterruptedException`, **восстановить флаг** (`Thread.currentThread().interrupt()`) и выйти из цикла. Если `Thread.interrupted()` стал `true` — тоже выйти. **Никаких пустых catch**.

Ожидаем: воркер корректно остановился и закончил работу при прерывании

Проверка работы

- Поднимите `LoopWorker("worker-1")`, дайте поработать 1.5–2 с, затем `stopAsync()` → поток должен **закончить сам** (восстанавливая флаг после `InterruptedException`).

3) `BackgroundTicker` — демон-тикер

Роль: фоновая печать тиков, **daemon**.

Методы/поведение:

- `static Thread startDaemon(String name, int periodMillis)` — создаёт `Thread`, `setDaemon(true)`, раз в `periodMillis` логирует «tick».
- Для демонстраций: запустите 2–3 user потока

Ожидаем: JVM завершится, когда все **user-потоки** завершены — даже если демон не докрутил цикл.

Проверка работы:

- Запустите `BackgroundTicker.startDaemon("ticker", 200)`.
- Параллельно поднимите ещё 1–2 user-потока (короткие вычисления 1–3 с).
- Покажите, что JVM завершилась **без** ожидания демона.

4) `VolatileFlagDemo` — флаг видимости (без атомарности)

Роль: показать, что **volatile** гарантирует видимость, но не атомарность сложных операций.

Методы/поведение:

- `private volatile boolean running;` — флаг «работать/останавливаться».

Методы:

- `void start()` — запускает поток, который делает лёгкую работу в цикле `while (running)` `{ /* work */ }`.
- `void stop()` — устанавливает `running=false`.

Ожидаем: без `volatile` поток может «не заметить» изменение (в демонстрационных целях можно временно убрать модификатор чтобы увидеть ошибку, как поток бесконечно крутится).

Проверка работы:

- Реализуйте `VolatileFlagDemo` со старт/стоп.
- Проверьте, что при `volatile` поток корректно «видит» `running=false`.

- `volatile` = видимость, не атомарность.

5) `MutexCounterDemo` — критическая секция на `synchronized`

Роль: короткая иллюстрация `mutex`: без синхронизации — гонка, с `synchronized` — корректно.

Интерфейс:

- `interface Counter`
 - `void inc();`
 - `long value();`

Реализации:

- `UnsafeCounter` — без синхронизации (`value++`).
- `SynchronizedCounter` — инкремент в `synchronized` -блоке.

Метод запуска:

- `static void runRace(int threads, int itersPerThread)` — создаёт N потоков, каждый делает M инкрементов; печатает ожидаемое `N*M` и фактическое.

Ожидаем: `Unsafe` даст сбой, `synchronized` — корректно (медленнее).

Проверка работы:

- Сравните `UnsafeCounter` и `SynchronizedCounter` на `runRace(8, 100_000)` ;
- Выведите `expected` , `actual`
- **синхронизация даёт mutual exclusion**, а без неё — гонка.



Мини-чеклист, что все работает верно

- ☐ **Java 21**, один `main` под каждую небольшую задачу-демо
- ☐ Логи содержат имена потоков и состояния (`getState()` в нужных местах).
- ☐ При `interrupt()` вы восстанавливаете флаг и выходите из цикла.
- ☐ **Daemon** запущен; приложение завершается без явной остановки демона.
- ☐ `volatile` используется **только** как флаг видимости (не для инкремента).
- ☐ Сравнение `Unsafe` vs `synchronized` показало гонку и её устранение.

☐ Нет «busy loop» без пауз — используйте `sleep()` /ожидания.

✓ Проверьте свои реализации

1. **Порядок непредсказуем** в `StartVsRunDemo` (лог показывает разные имена потоков и перемешивание).
2. `LoopWorker` корректно завершает цикл при `interrupt()` ; нет заглатывания `InterruptedException` , флаг восстановлен.
3. **Даemon-тикер** не удерживает JVM (демо завершения видно по логам).
4. `VolatileFlagDemo` : при `volatile` поток **гарантированно** останавливается по установке флага.
5. `MutexCounterDemo` : `UnsafeCounter` даёт `actual < expected` ; `SynchronizedCounter` — `actual == expected` .

💡 Подсказки

- `run()` **не** запускает новый поток; `start()` — да. Это обязательный пункт демонстрации.
- `volatile` **не делает** `x++` атомарным — не используйте его для счётчиков. Для демонстрации атомарности берём `synchronized` .
- Демон-потоки не ждут graceful-shutdown — поэтому и показываем «обрыв» тиков в конце.
- Не глотайте `InterruptedException` : восстановите флаг, завершитесь. Это базовая культура работы с потоками.